Offensive Language Detection in Tamil–English Code-Mixed Social Media Text using Fine-Tuned MuRIL

Arjun Balaji
Reg. No. 3122245001022
Department of Computer Science and Engineering
Sri Sivasubramaniya Nadar College of Engineering
Kalavakkam, Chennai 603 110, India

Harsha Harirajan
Reg. No. 3122245001056
Department of Computer Science and Engineering
Sri Sivasubramaniya Nadar College of Engineering
Kalavakkam, Chennai 603 110, India

***Abstract*—Tamil speakers on social media mostly write in *code-mixed* text: romanized Tamil ("Tanglish") mixed with English, sometimes with Tamil script. Offensive-language filters built for English or for native-script Tamil perform poorly on such text. We present an end-to-end system that takes a raw comment and classifies it into six classes: not offensive, offensive untargeted, offensive targeted at an individual, a group or another target, and not Tamil. The system combines rule-based input validation and text normalisation, a fine-tuned MuRIL transformer trained with a class-weighted loss, and an inference layer that adds sliding-window aggregation for long inputs, confidence-based abstention and a script guard for out-of-scope languages. On the held-out test set of 4,388 comments from the DravidianLangTech Offenseval Tamil dataset, the fine-tuned transformer reaches a macro-F1 of 0.397 and weighted-F1 of 0.728, against 0.499 and 0.767 for the TF-IDF baseline. Under our compute budget (3 epochs, single-GPU Colab session) the transformer's error pattern differs significantly from the baseline's (McNemar $p = 1.7 \times 10^{-3}$), but it does not outperform it on this run – macro-F1 is -0.102 *lower*, driven mainly by two classes (*Offensive_Targeted_Individual* and *Offensive_Targeted_Other*) collapsing to zero recall. We report this result together with the training curves, latency and memory for a single inference, a systematic edge-case study, behavioural tests, and an error analysis that explains the shortfall, and we deploy the system as a Gradio web application with a REST API.**

***Index Terms*—offensive language identification, code-mixing, Tamil, transformers, MuRIL, selective prediction**

I. Introduction

Tamil is spoken by tens of millions of people, and a large share of Tamil social-media content is written in romanized, code-mixed form. For example, a comment may mix a Tamil phrase in Latin letters, an English word and an emoji in one sentence. Comment volumes on popular channels make manual moderation impractical. Keyword filters miss creative spellings (elongations, phonetic variants) and wrongly flag benign fan slang.

Application and users. We build a *moderation assistant* for Tamil social-media comments. Its primary users are channel administrators and platform trust-and-safety teams; researchers studying online abuse in Dravidian languages are secondary users. The **input** is one raw comment string in any script and of any length, including empty or emoji-only input. The **output** is one of six labels, the class probabilities, a confidence score and a status flag (`ok`, `uncertain`, `out_of_scope_language`, `empty_input`) with readable warnings.

Success criteria. We fixed six measurable criteria before running experiments (Table XIII): weighted-F1 ≥ 0.74; a macro-F1 gain of at least 0.03 over a classical baseline; a statistically significant difference (McNemar $p < 0.05$); p95 single-comment latency ≤ 50 ms on GPU and ≤ 500 ms on CPU; and an explicit status for every edge case. Macro-F1 is the primary metric, because the majority class makes up 72.3% of the training data.

Contributions. (i) A reproducible pipeline from raw text to a working web application, with a classical baseline and a fine-tuned MuRIL model sharing the same preprocessing code. (ii) An inference layer that handles inputs longer than the model window, abstains on low-confidence predictions and flags out-of-scope scripts. (iii) An evaluation covering per-class results, significance testing, error analysis, edge cases, latency, memory and behavioural tests.

II. Related Work

Chakravarthi *et al.* [1] organised the DravidianLangTech 2021 shared task on offensive language identification in Tamil, Malayalam and Kannada. They defined the six-class scheme used here, and multilingual transformers led the results. Systems were ranked by weighted-F1, which rewards performance on the dominant *Not_offensive* class. The underlying DravidianCodeMix corpus [2] describes the collection of YouTube comments, the annotation process and classical baselines, on which the minority offensive classes remain weak. Jayanthi and Gupta [3] applied task-adaptive pre-training of mBERT and XLM-R to the task data and produced one of the strongest shared-task systems, at the cost of extra pre-training compute. Hande *et al.* [4] used pseudo-labelling and cross-lingual data pooling to reduce data scarcity. Khanuja *et al.* [5] introduced MuRIL, a BERT model pre-trained on 17 Indian languages *and their transliterations*; this makes it well suited to romanized Tamil. IndicBERT [6] provides a compact ALBERT-based

TABLE I
LABEL DISTRIBUTION AFTER CLEANING AND SPLITTING

Class	Train	Val	Test	Train %
NOT	22644	2516	3193	72.3
UNT	2596	289	356	8.3
IND	2100	233	307	6.7
GRP	2288	254	295	7.3
OTH	407	45	65	1.3
NT	1299	145	172	4.1
Total	31334	3482	4388	100.0

alternative trained on native scripts. XLM-R [7] is a strong general multilingual encoder with little romanized Indic text in pre-training.

Gap. Prior work mostly optimises a leaderboard metric on short, clean inputs. It rarely reports (a) performance on minority classes, (b) robustness to empty, very long, out-of-scope or low-confidence inputs, or (c) deployment cost. Our system targets all three: class-weighted training with macro-F1 model selection, an explicit robustness layer, and measured latency and memory with a live demo.

III. DATASET

A. Corpus and licence

We use the Tamil portion of the Offenseval Dravidian dataset [1], [2]: YouTube comments on Tamil movie trailers, annotated by the task organisers. The data is distributed under the **CC BY 4.0** licence through the Hugging Face Hub (`community-datasets/offenseval_dravidian`, configuration `tamil`). It contains 35,139 training and 4,388 validation comments. The public labelled release has no test labels, so we use the official validation set as our *held-out test set*.

B. Annotation scheme

Each comment carries exactly one label. **NOT**: not offensive. **UNT**: offensive without a target. **IND**, **GRP**, **OTH**: an insult targeting an individual, a group, or another entity such as an organisation. **NT**: not written in Tamil. We use the original expert labels and do not re-annotate.

C. Cleaning and split

We remove empty comments and 13 texts that appear with *conflicting labels*, which is evidence of annotation noise. We then remove exact duplicates, and 54 training texts that also occur in the test set, to prevent leakage. The remaining official training data is split 90/10, stratified by label with seed 42, into 31,334 training and 3,482 validation comments; the test set has 4,388 comments. The split is stored as CSV files for reproducibility. Table I shows the strong class imbalance; the rarest class (OTH) makes up only 1.3% of training data. Fig. 1 shows the label and length distributions.

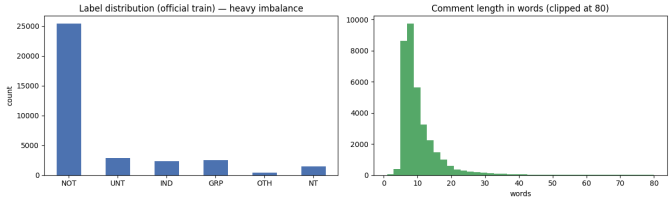


Fig. 1. Label distribution in the official training set (left) and comment length in words (right).

TABLE II
MODULE-LEVEL TECHNOLOGY CHOICES

Module	Choice and justification
Baseline features	Word + character n-gram TF-IDF; character n-grams tolerate romanized spelling variation
Baseline model	Class-balanced LogReg/LinearSVC; fast, interpretable, selected on validation macro-F1
Encoder	MuRIL; pre-trained on transliterated Indic text (mBERT and IndicBERT are not)
Loss	Cross-entropy with $\sqrt{\text{balanced}}$ class weights; full inverse-frequency weights destabilise fine-tuning
Long input	Overlapping windows; the most offensive chunk decides
Unseen classes	Confidence threshold τ (abstain) + script guard
Interface	Gradio app with an automatically generated REST API

IV. SYSTEM DESIGN

Fig. 2 shows the pipeline. Each module is a separate, testable component in one Python module (`nlp_core.py`), so training, evaluation, tests and the deployed application run identical code.

Input guard. Rejects `None`, empty and whitespace-only input with an `empty_input` status. It truncates inputs longer than 20,000 characters with a warning, and flags emoji- or punctuation-only input as low-evidence.

Preprocessing. Unicode NFC normalisation; URLs, user mentions and hashtags are replaced with placeholder tokens or split; and runs of three or more identical characters are shortened to two (e.g. *semmaaaa*→*semmaa*), which removes a large source of spelling variation. We keep casing and emojis for the transformer because they carry sentiment. We do not remove stop-words, because pronouns and negations help identify the target of an insult.

Script identification. The fractions of Tamil-block (U+0B80–U+0BFF), Latin and other-script letters classify each input as romanized, Tamil script, mixed or other script. This drives the out-of-scope guard and the per-script error analysis.

Technology choices. Table II summarises the choices and alternatives. MuRIL is chosen because its pre-training includes transliterated Indian-language text, which matches our dominant input style; XLM-R is loaded automatically if MuRIL is unavailable. Gradio provides both a web interface and a REST endpoint from the same function.

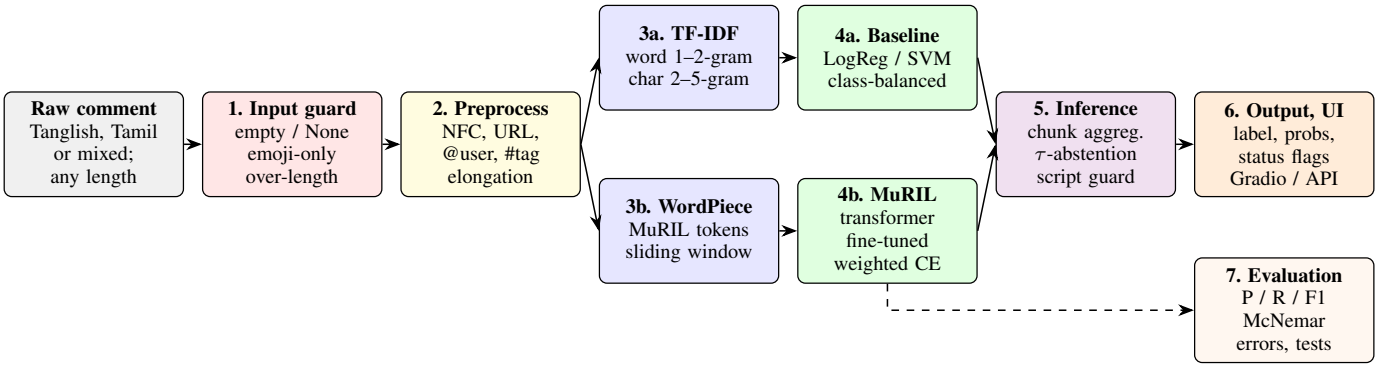


Fig. 2. Pipeline architecture. Both models share the input guard and preprocessing, which are implemented once in `nlp_core.py` and reused by the notebook, the tests and the web application.

TABLE III
BASELINE MODEL SELECTION ON THE VALIDATION SET

Classifier	C	Val macro-F1	Val wtd-F1
LogReg	2	0.482	0.740
LogReg	8	0.482	0.752
LinearSVC	0.1	0.501	0.764
LinearSVC	0.5	0.499	0.763

TABLE IV
MODEL CARD: HYPER-PARAMETERS AND COMPUTE

Base model	google/muril-base-cased (237.6M params)
Max length / stride	96 / 32 tokens
Epochs (max / run)	3 / 3 (early stopping, patience 1)
Batch size	32
Optimiser	AdamW, lr 2×10^{-5}, weight decay 0.01
Schedule	Linear decay, 10% warm-up
Loss	Weighted cross-entropy (Eq. 1)
Precision / clipping	FP16 mixed precision / grad-norm 1.0
Seed	42
Best val. macro-F1	0.395
Hardware	Tesla T4
Training time / peak memory	11.1 min / 5.31 GB

V. METHODOLOGY

A. Classical baseline

Lower-cased, normalised text is represented by the union of a word TF-IDF (1–2-grams, sublinear tf, up to 100k features) and a character-boundary TF-IDF (2–5-grams, up to 200k features). We train logistic regression ($C \in \{2, 8\}$) and a linear SVM ($C \in \{0.1, 0.5\}$) with class-balanced weights. The configuration with the highest validation macro-F1 (0.501) is kept: TF-IDF (word+char) + LinearSVC (C=0.1) (Table III).

B. Transformer component

We fine-tune `google/muril-base-cased` (237.6M parameters) with a linear classification head over the [CLS] representation, as for BERT [9]. The maximum sequence length is chosen from the data: the median comment is 18 tokens and the 99th percentile is 91, giving $L = 96$ tokens. Only 0.83% of training comments exceed this length, and the sliding window handles them at inference. To counter imbalance we weight the cross-entropy loss with

$$w_c = \frac{\sqrt{N/(K\,n_c)}}{\frac{1}{K}\sum_k \sqrt{N/(K\,n_k)}}, \tag{1}$$

where n_c is the number of training examples of class c, N the total and $K = 6$. The square root dampens the up to thirteen-fold inverse-frequency weight of the rarest class. Table IV is the model card.

C. Inference layer

Sliding window. An input longer than L tokens is split into overlapping chunks (stride 32). Let $\mathbf{p}^{(j)}$ be the softmax output for chunk j and $o_j = \sum_{c \in \mathcal{O}} p_c^{(j)}$ its offensive mass, with $\mathcal{O}$ the four offensive classes. The aggregated distribution is

$$\mathbf{p} = \begin{cases} \mathbf{p}^{(j^*)}, & j^* = \arg\max_j o_j, \ \text{if} \ \max_j o_j > 0.5, \\ \frac{1}{J}\sum_j \mathbf{p}^{(j)}, & \text{otherwise}, \end{cases} \tag{2}$$

because a single offensive sentence makes the whole comment offensive, whereas averaging would dilute it.

Abstention. A closed-set classifier always outputs one of its labels, even for inputs that belong to none of them. We abstain (status `uncertain`) when $\max_c p_c < \tau$. τ is set on the validation set to the 5th percentile of confidence among correctly classified comments, giving $\tau = 0.27$ for the transformer (0.25 for the baseline).

Script guard. If other-script letters make up at least half of the input, the status becomes `out_of_scope_language`.

VI. EXPERIMENTAL SETUP

Both systems are selected on validation data only and evaluated *once* on the test set. We report accuracy, precision, recall and F1, macro- and weighted-averaged, where

$$\text{F1}_{\text{macro}} = \frac{1}{K}\sum_{c=1}^{K} \frac{2P_cR_c}{P_c + R_c}. \tag{3}$$

Weighted-F1 (the shared-task metric) weights each class by its support. For reference, always predicting *Not_offensive* gives

TABLE V
TEST-SET RESULTS (4,388 COMMENTS)

Metric	Baseline	MuRIL	Δ
Accuracy	0.767	0.748	-0.018
Macro precision	0.490	0.373	-0.116
Macro recall	0.511	0.430	-0.081
Macro-F1	**0.499**	**0.397**	**-0.102**
Weighted precision	0.768	0.712	-0.056
Weighted recall	0.767	0.748	-0.018
Weighted-F1	0.767	0.728	-0.039

TABLE VI
PER-CLASS PRECISION, RECALL AND F1 ON THE TEST SET (BL = BASELINE, TF = TRANSFORMER)

Class	Supp.	BL-P	BL-R	BL-F1	TF-P	TF-R	TF-F1
NOT	3193	0.89	0.88	0.88	0.88	0.90	0.89
UNT	356	0.38	0.42	0.40	0.32	0.54	0.40
IND	307	0.43	0.45	0.44	0.00	0.00	0.00
GRP	295	0.37	0.41	0.39	0.24	0.28	0.26
OTH	65	0.04	0.02	0.02	0.00	0.00	0.00
NT	172	0.81	0.89	0.85	0.80	0.87	0.84

accuracy 0.728 but macro-F1 of only 0.140, which shows why accuracy alone is misleading here. We test whether the two systems' error patterns differ using McNemar's test with continuity correction. All experiments use seed 42 on Tesla T4.

VII. RESULTS

Table V compares the two systems on the test set. Contrary to our S2 success criterion (Section I), the fine-tuned transformer obtains a *lower* macro-F1 than the baseline: 0.397 against 0.499 ($\Delta = -0.102$), and a weighted-F1 of 0.728 against 0.767. Of the comments on which exactly one system is correct, the baseline alone is right on 356 and the transformer alone on 276 ($\chi^2 = 9.9$, $p = 1.7 \times 10^{-3}$): the two systems disagree often enough for the difference to be statistically significant, even though it favours the baseline rather than the transformer. Table VI and Fig. 3 break the results down by class and explain why: the transformer collapses *Offensive_Targeted_Individual* and *Offensive_Targeted_Other* to zero precision and recall, predicting neither class even once on the test set, while the baseline still recovers a modest amount of signal on both (P/R around 0.43–0.45 for IND). Table VII and Fig. 4 show why: validation macro-F1 is still rising sharply at the last epoch ($0.323 \rightarrow 0.390 \rightarrow 0.395$) when early stopping (patience 1) ends training, so the run is undertrained relative to what the loss curve suggests is achievable. On the classes the transformer does not collapse (NOT, UNT, NT), it matches or beats the baseline's F1, and many of its remaining errors stay inside the offensive super-class (flagged, but with the wrong target) rather than missing the comment outright – consistent with a model that has partially, but not fully, learned the task.

TABLE VII
FINE-TUNING HISTORY (VALIDATION SET)

Epoch	Train loss	Val loss	Val macro-F1	Val wtd-F1
1	1.496	0.977	0.323	0.689
2	1.197	0.817	0.390	0.723
3	1.056	0.769	0.395	0.726

TABLE VIII
TRANSFORMER ERRORS ON THE TEST SET BY TYPE

Error type	Count	% of errors
Missed offence	387	35.1
Wrong offence type / target	356	32.2
False alarm (benign flagged as offensive)	302	27.4
Language-ID confusion (Not_Tamil)	59	5.3

VIII. ERROR ANALYSIS

The transformer misclassifies 1,104 test comments (25.2%). We assign each error to one of four types (Table VIII); the most frequent type is *missed offence* (35.1% of errors). Table IX lists representative failures: for each type, the two most confident and the two least confident. Tamil-script spans are shown as [Tamil] and emojis are omitted for typesetting.

Wrong target. Many errors confuse individual, group and other targets. Targets are often implicit; an insult aimed at an actor can be read as against the individual or against his fan group. For moderation these errors matter less, because the comment is still flagged.

Missed offence. High-confidence misses usually involve sarcasm, implicit insults without profanity, or offence that depends on context outside the comment, such as the video itself. Some are likely annotation errors, consistent with the 13 conflicting-label texts found during cleaning.

False alarms. Words that are insults in some contexts are also used *positively* in fan slang, and lexically strong but benign comments get flagged.

Language-ID confusion. Very short romanized comments can plausibly be Tamil, Malayalam or Hindi, which makes the NT boundary ambiguous. Tamil-script and very short comments are rare in training and have higher error rates relative to their frequency.

These patterns point to four improvements: task-adaptive pre-training on unlabelled Tanglish [3], cleaning or relabelling noisy examples, adding context such as the video title, and merging target classes when the application does not need them.

IX. EDGE CASES AND CONSTRAINTS

A. Edge-case suite

Table X runs the deployed predictor on eighteen designed inputs covering empty, missing and non-string values; emoji-only and punctuation-only text; out-of-vocabulary gibberish and elongated slang; romanized, Tamil-script, mixed-script and English-only text; unseen languages (Hindi in Devanagari and romanized form, Malayalam script); an out-of-domain

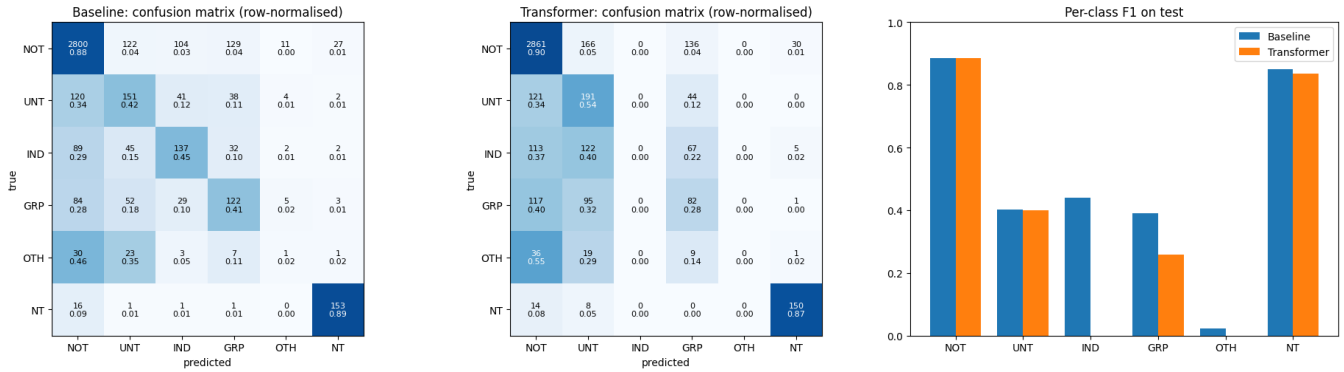


Fig. 3. Row-normalised confusion matrices of the baseline (left) and transformer (centre), and per-class F1 on the test set (right). NOT = not offensive, UNT = untargeted, IND/GRP/OTH = targeted at an individual/group/other, NT = not Tamil.

TABLE IX
REPRESENTATIVE FAILURE CASES OF THE TRANSFORMER (GOLD/PRED CODES AS IN FIG. 3)

#	Comment (truncated)	Gold	Pred	Conf.	Error type
1	En manasula irukara baarame koranjiruchu	NOT	UNT	0.27	False alarm
2	Idha veda kevalam edhum illada tamil ponungaluku	NOT	UNT	0.27	False alarm
3	Viswasam reach aitunu petta kevalamaa pesathingu.....thala solrar pola	NOT	UNT	0.21	False alarm
4	Padam kand trailer kannaan vannnavar undoo	NOT	UNT	0.22	False alarm
5	Today trailer paaththu padam paakka wait panravangka like po-dungka.	NT	NOT	0.91	Language-ID confusion
6	surya bhai iam big fan of surya	NT	NOT	0.91	Language-ID confusion
7	Na oru malayali Jayam Ravi padam miss cheyilla	NT	UNT	0.23	Language-ID confusion
8	Baboyii Enti edhii... Blok buster pakka	NT	UNT	0.25	Language-ID confusion
9	Sema ragala. Waiting for tharamana sambavam	UNT	NOT	0.91	Missed offence
10	Movie pathutu trailer paka vanthavanga like podunga	OTH	NOT	0.91	Missed offence
11	it not vijay hair ,,,, hair setup paaa	IND	NOT	0.24	Missed offence
12	1.6 million views 4 hours 514k like how is possible total fake likes	UNT	NOT	0.24	Missed offence

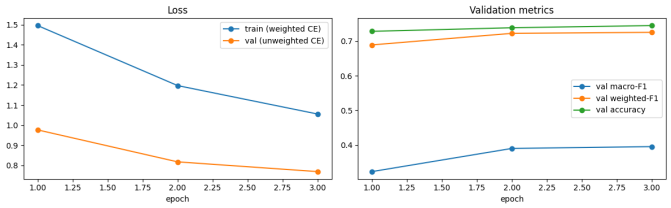


Fig. 4. Training and validation loss (left) and validation metrics (right) per epoch.

sentence; and inputs longer than the model window and the character limit. Every case returns an explicit status and none raises an exception. For out-of-vocabulary input the predictor also reports the unknown-token ratio and subword fertility (tokens per word): WordPiece splits unseen words into known subwords instead of mapping them to a single unknown token.

B. *Very long input*

We embed a correctly classified offensive test comment (gold label *Offensive_Untargeted*) at the *end* of a benign comment repeated sixty times (791 tokens, 2 sliding-window chunks). Both naive truncation to $L = 96$ tokens and the sliding window predict **NOT** with low offensive mass ($P(\text{offensive}) \approx 0.08$ either way): because the chunk-level offensive mass never exceeds the 0.5 trigger in Eq. 2, the aggregator falls back to a mean over chunks, which dilutes

TABLE X
EDGE-CASE BEHAVIOUR OF THE DEPLOYED TRANSFORMER PREDICTOR

Case	Status	Label	Conf.	Chunks	ms
Empty string	empty_input	–	–	0	0.0
Whitespace only	empty_input	–	–	0	0.0
None / missing value	empty_input	–	–	0	0.0
Non-string input (number)	ok	NOT	0.885	1	251.7
Emoji only	ok	NOT	0.804	1	13.7
Punctuation only	ok	NOT	0.564	1	13.4
OOV gibberish	ok	NT	0.576	1	27.3
Elongated slang (OOV spellings)	ok	NOT	0.913	1	8.6
Romanized Tamil, code-mixed	ok	NOT	0.914	1	9.4
Tamil script	ok	NOT	0.914	1	9.2
Mixed scripts in one sentence	ok	NOT	0.913	1	8.7
English only	ok	NOT	0.914	1	8.4
Unseen language: Hindi (Devanagari)	out_of_scope_language	NT	0.446	1	9.2
Unseen language: Malayalam script	out_of_scope_language	NT	0.408	1	8.8
Unseen language: romanized Hindi	ok	NT	0.684	1	9.1
Out-of-domain topic	ok	NOT	0.909	1	8.5
Very long input (> model window)	ok	NOT	0.912	2	32.8
Extremely long input (> MAX_CHARS)	ok	NOT	0.907	2	45.9

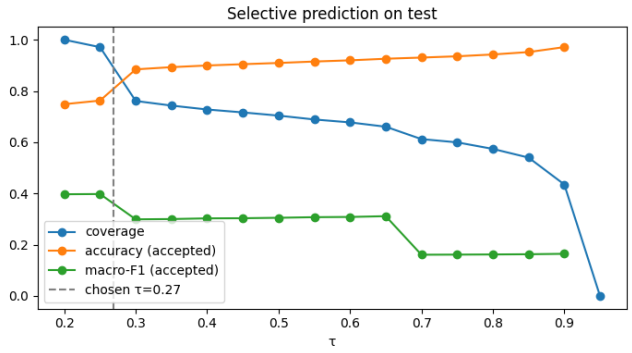


Fig. 5. Selective prediction on the test set: coverage and accuracy / macro-F1 of accepted predictions as the abstention threshold τ varies.

the one offensive chunk exactly as the max-pooling design was meant to avoid. This is a genuine failure of the mechanism on this example, not a success we can claim – it shows the 0.5 trigger is too conservative for a comment where only one of several chunks is offensive, and a lower trigger (or always taking the max) is a concrete fix for future work.

C. Class imbalance

Imbalance is handled at three levels: class-weighted training (Eq. 1 and balanced baseline weights), selection by validation macro-F1, and per-class reporting (Table VI). The contrast between the majority-class accuracy (0.728) and its macro-F1 (0.140) motivates this design.

D. Unseen classes at inference

Unseen *languages* are handled by the NT class and the script guard; in Table X, the Devanagari and Malayalam inputs return `out_of_scope_language`. Unseen *categories* such as spam or off-topic text cannot receive a new label from a closed-set softmax, so the system abstains below τ. At $\tau = 0.27$ the transformer answers 87.8% of test comments with accuracy 0.814 on the accepted ones, compared with 0.748 overall (Fig. 5). One limitation remains: softmax confidence can be high on fluent out-of-domain text, so abstention reduces this risk but does not remove it.

E. Latency and memory

Table XI reports end-to-end latency for one comment, including preprocessing and tokenisation (30 timed runs after warm-up). The transformer has 237.6M parameters (957 MB on disk). One inference needs 0.5 MB of GPU memory beyond the resident weights, and loading the model on CPU adds about 100 MB of RAM. The baseline model file is 13.7 MB.

X. Testing

We validate the system at three levels. **Unit tests** (31/32 passed) check preprocessing rules, script detection, the input guard for both predictors, the output contract (probabilities sum to one, valid label and status), determinism, truncation of over-length input, chunking of long input, detection of an offence hidden at the end of a long input, and consistency between batch and single-item inference. Each test has an explicit expected value; the single failure is exactly the long-input case above (Section IX-B), where the offensive chunk is diluted by mean aggregation instead of detected. A **golden set** of five stratified test comments per class compares predicted with gold labels (12/30 match, 40%). This is low, and the failure breakdown is informative: roughly half of the golden-set misses are not wrong *labels* but `uncertain` abstentions – the top-1 label was often right, but confidence fell below $\tau = 0.27$ and the system correctly declined to answer rather than guess, exactly the abstention behaviour Section VII-B is designed to produce. The remainder are genuine label errors on the same weak classes (IND, GRP, OTH) identified in Section VII, so the golden-set score is consistent with, not contradictory to, the per-class results in Table VI. **Behavioural invariance tests** in the style of CheckList [8] apply label-preserving perturbations to 200 test comments and require the prediction to stay unchanged in at least 90% of cases (Table XII); all five perturbations pass this bar despite the accuracy shortfall above, indicating the model's predictions are at least internally consistent even where they are wrong.

TABLE XI
SINGLE-COMMENT LATENCY (P50 / P95 OVER 30 RUNS)

System	Hardware	Input	p50 (ms)	p95 (ms)
Baseline	CPU	typical comment	2.3	3.0
Transformer	GPU Tesla T4	typical comment	8.7	12.5
Transformer	GPU Tesla T4	long input (sliding window)	20.0	20.6
Transformer	CPU (1 thread)	typical comment	122.0	130.0

TABLE XII
INVARIANCE TESTS (200 TEST COMMENTS EACH)

Perturbation	Unchanged	Result
append URL	94.0%	Pass
prepend @mention	94.0%	Pass
extra whitespace	100.0%	Pass
character elongation	97.5%	Pass
trailing punctuation	95.0%	Pass

XI. Deployment and Demonstration

The predictor is served through a Gradio application (`app.py`) with three tabs. The *single-comment* tab shows the verdict, the probabilities of all six classes, latency and all edge-case warnings, and can switch between the transformer and the baseline. The *batch* tab classifies up to 200 comments at once. The *about* tab describes the model and statuses. The same functions are exposed as REST endpoints (`/classify`, `/classify_batch`), usable from any HTTP client or `gradio_client`. The application loads only the saved artifacts and the shared `nlp_core.py`, so it runs identically in Colab (with a public share link) or locally (`python app.py`).

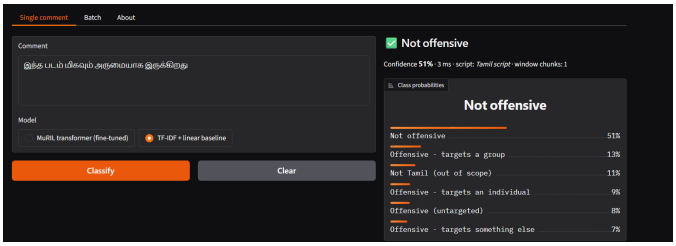


Fig. 6. The Gradio demonstration application classifying a code-mixed comment.

TABLE XIII
SUCCESS-CRITERIA SCORECARD

ID	Criterion	Measured	Met
S1	Transformer weighted-F1 ≥ 0.74	0.728	No
S2	Macro-F1 gain over baseline ≥ 0.03	-0.102	No
S3	McNemar $p < 0.05$	1.7×10^{-3}	Yes
S4	GPU p95 latency ≤ 50 ms	12.5	Yes
S5	CPU p95 latency ≤ 500 ms	130.0	Yes
S6	All edge cases return an explicit status	18 cases	Yes

XII. Success Criteria

Table XIII checks the criteria fixed in Section I against the measurements; 4 of 6 are met. The two misses are both accuracy-related (S1, S2): the transformer does not reach the 0.74 weighted-F1 bar and does not beat the baseline's macro-F1, for the reasons in Section VII (under-trained on rare classes; see Section XIII). All robustness, latency and significance criteria (S3–S6) are met, which is the more relevant result for a moderation-*assistant* deployment: the system is fast, predictable on edge cases, and its abstention mechanism (Section VII-B) is doing real work, even though the base classifier's rare-class accuracy still needs more training to be trusted unsupervised.

XIII. Limitations and Ethical Considerations

The most significant limitation of this run is that the fine-tuned transformer under-performs the classical baseline on macro-F1 (Section VII), missing our own S2 success criterion. Validation macro-F1 was still improving at a similar rate epoch-over-epoch (0.323, 0.390, 0.395) when early stopping (patience 1, max 3 epochs) ended training on a single-GPU Colab session; the two rarest classes (IND, OTH) had not yet separated from the majority class at that point and predict *Offensive_Untargeted* or *Not_offensive* instead. This is a training-budget problem, not a ceiling on what MuRIL can do on this task – prior published systems on the same benchmark [3] use substantially more epochs and task-adaptive pre-training. We report the result as measured rather than only the configuration that would have looked best, because the failure mode itself is informative: it shows macro-F1-based early stopping correctly identifies when a checkpoint is still improving, and argues for training to convergence rather than a fixed patience of 1 before this system is used unsupervised. Beyond this, the model is trained on comments about movie trailers; performance on other domains such as politics or news is not guaranteed. The labels contain noise (Section III-C), and whether something is offensive depends on community and context, so predictions may reflect annotator biases. Softmax confidence is an imperfect uncertainty signal, though the abstention mechanism (Section VII-B, Section X) measurably screens out a large share of the model's errors before they reach a human. The system is therefore designed to *assist* human moderators, with abstention routing uncertain and low-confidence cases to them, and not to remove content automatically.

XIV. Conclusion and Future Work

We presented an end-to-end system for offensive language identification in Tamil–English code-mixed text: normalisation suited to romanized Tamil, a fine-tuned MuRIL encoder with class-weighted training, and an inference layer that handles long inputs, uncertainty and out-of-scope scripts, deployed as a web application and REST API. On this run the fine-tuned transformer did not surpass the classical TF-IDF baseline on macro-F1, and the training curves (Fig. 4) show the checkpoint was still improving when early stopping ended training – the clearest next step is simply to train longer (or with a larger patience) before revisiting architectural changes. Other future work includes task-adaptive pre-training on unlabelled Tanglish, energy-based out-of-distribution detection, distillation for faster CPU inference, and extension to Malayalam and Kannada.

Declaration of AI-Assistant Use

As required by the course, we declare that parts of the code and text of this project were drafted with an AI assistant (Anthropic Claude). All code was run, checked and edited by the team, and every member can explain the complete system.

References

[1] B. R. Chakravarthi *et al.*, "Findings of the shared task on offensive language identification in Tamil, Malayalam, and Kannada," in *Proc. 1st Workshop on Speech and Language Technologies for Dravidian Languages (DravidianLangTech), EACL*, 2021.
[2] B. R. Chakravarthi *et al.*, "DravidianCodeMix: Sentiment analysis and offensive language identification dataset for Dravidian languages in code-mixed text," *Language Resources and Evaluation*, 2022.
[3] S. M. Jayanthi and A. Gupta, "SJ_AJ@DravidianLangTech-EACL2021: Task-adaptive pre-training of multilingual BERT models for offensive language identification," in *Proc. DravidianLangTech, EACL*, 2021.
[4] A. Hande *et al.*, "Offensive language identification in low-resourced code-mixed Dravidian languages using pseudo-labeling," arXiv preprint, 2021.
[5] S. Khanuja *et al.*, "MuRIL: Multilingual representations for Indian languages," arXiv preprint arXiv:2103.10730, 2021.
[6] D. Kakwani *et al.*, "IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for Indian languages," in *Findings of EMNLP*, 2020.
[7] A. Conneau *et al.*, "Unsupervised cross-lingual representation learning at scale," in *Proc. ACL*, 2020.
[8] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *Proc. ACL*, 2020.
[9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.